

PART I

IMAGE PROCESSING

INSTRUCTOR: DR. NGUYEN NGOC TRUONG MINH

SCHOOL OF ELECTRICAL ENGINEERING, INTERNATIONAL UNIVERSITY (VNU-HCMC)

LECTURE XI – COLOR IMAGE PROCESSING

INSTRUCTOR: DR. NGUYEN NGOC TRUONG MINH

SCHOOL OF ELECTRICAL ENGINEERING, INTERNATIONAL UNIVERSITY (VNU-HCMC)

Ho Chi Minh City, Fall 2024

LECTURE CONTENT

- What are *the most important concepts and terms* related to color perception?
- What are *the main color models* used to represent and quantify color?
- How are color images represented in MATLAB?
- What is *pseudo-color image processing* and how does it differ from full-color image processing?
- How can monochrome image processing techniques be extended to color images?
- Chapter Summary – What we have learned?
- Problems

11.1 THE PSYCHOPHYSICS OF COLOR

- **Color perception** is a *psychophysical phenomenon* that combines two main components:
 1. The physical properties of light sources (usually expressed by their spectral power distribution (SPD)) and surfaces (e.g., their absorption and reflectance capabilities).
 2. The physiological and psychological aspects of the human visual system (HVS).

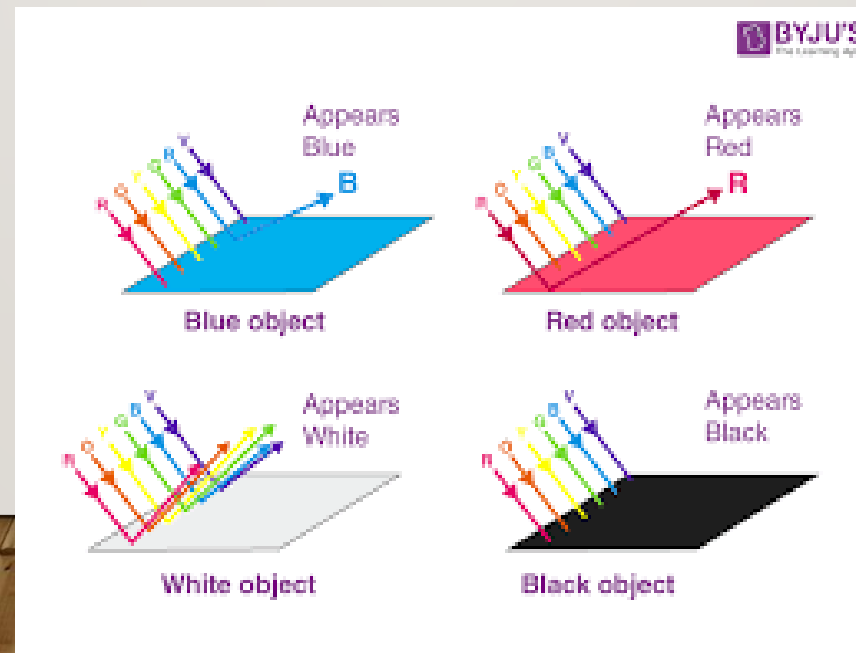
11.1.1 BASIC CONCEPTS

- The perception of color starts with *a chromatic light source*, capable of emitting electromagnetic radiation with wavelengths between *approximately 400 and 700 nm*. Part of that radiation reflects on the surfaces of the objects in a scene and the resulting reflected light reaches the human eye, giving rise to the sensation of color.
- An object that reflects light almost equally in all wavelengths within the visible spectrum is perceived as white, whereas an object that absorbs most of the incoming light, regardless of the wavelength, is seen as black.



11.1.1 BASIC CONCEPTS

- The perception of several shades of gray between pure white and pure black is usually referred to as *achromatic*. Objects that have more selective properties are considered *chromatic*, and the range of the spectrum that they reflect is often associated with a color name.



11.1.1 BASIC CONCEPTS

- For example, an object that absorbs most of the energy within the 565–590 nm wavelength range is considered yellow.
- A chromatic light source can be described by three basic quantities:
 - *Intensity (or Radiance)*: the total amount of energy that flows from the light source, measured in watts (W).
 - *Luminance*: a measure of the amount of information an observer perceives from a light source, measured in lumen (lm). It corresponds to the radiant power of a light source weighted by a spectral sensitivity function (characteristic of the HVS).
 - *Brightness*: the subjective perception of (achromatic) luminous intensity.

11.1.1 BASIC CONCEPTS

- **The human retina** (the surface at the back of the eye where images are projected) is coated with photosensitive receptors of two different types: *cones and rods*.
- Rods cannot encode color but respond to lower luminance levels and enable vision under darker conditions.
- Cones are primarily responsible for color perception and operate only under bright conditions. There are three types of cone cells (L cones, M cones, and S cones, corresponding to long (≈ 610 nm), medium (≈ 560 nm), and short (≈ 430 nm) wavelengths, respectively).

11.1.1 BASIC CONCEPTS

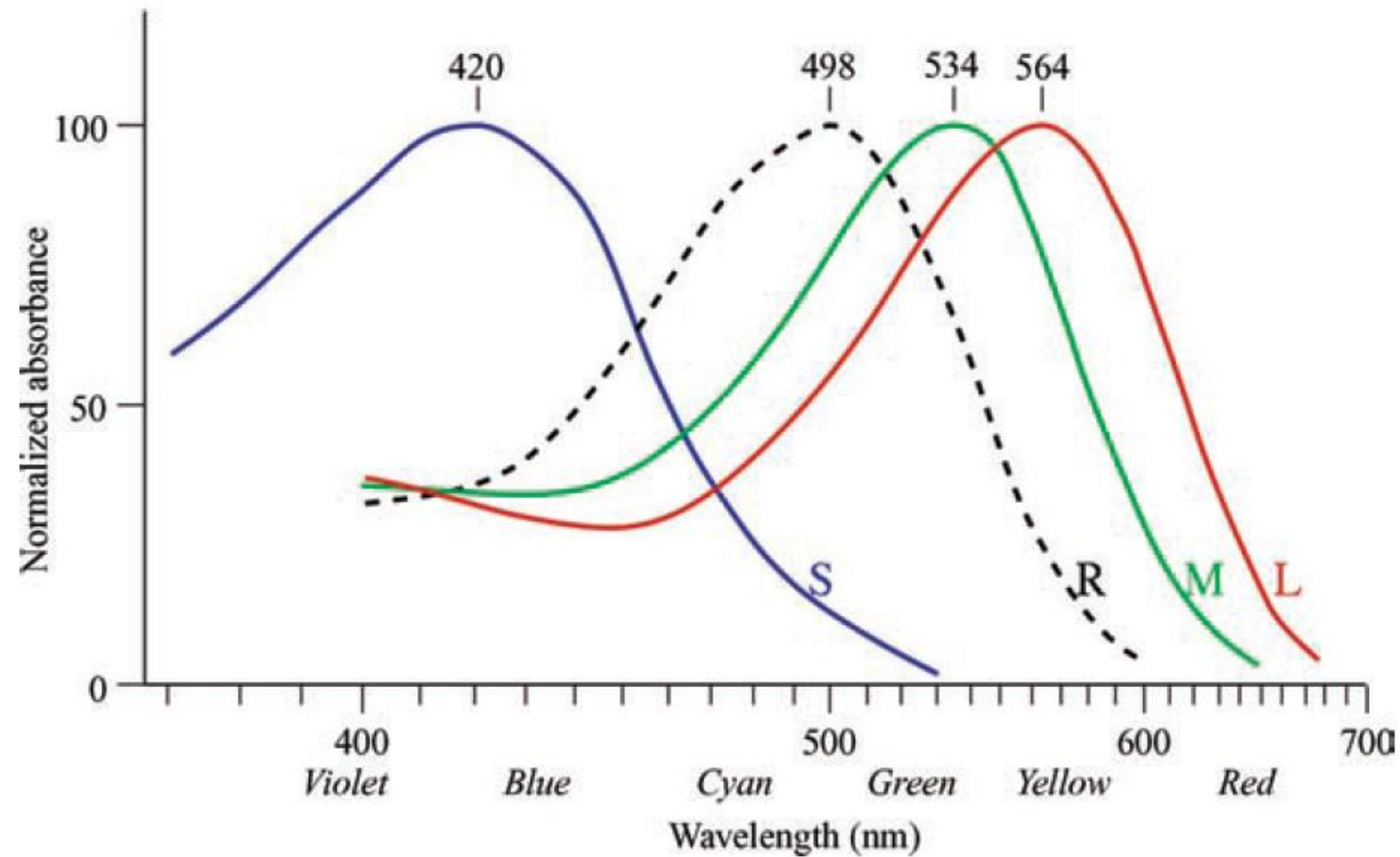


FIGURE 16.1 Spectral absorption curves of the short (S), medium (M), and long (L) wavelength pigments in human cone and rod (R) cells. Courtesy of Wikimedia Commons.

11.1.1 BASIC CONCEPTS

- *The existence of three specialized types of cones in the human eye was hypothesized more than a century before it could be confirmed experimentally by Thomas Young and his trichromatic theory of vision in 1802.*
- *The secondary colors of light, obtained by additive mixtures of the primaries, two colors at a time, are *magenta (or purple) = red + blue*, *cyan (or turquoise) = blue + green*, and *yellow = green + red* (Figure 11.2a).*

11.1.1 BASIC CONCEPTS

- For color mixtures using **pigments** (or *paints*), the primary colors are *magenta, cyan, and yellow* and the secondary colors are *red, green, and blue* (Figure 11.2b). It is important to note that for pigments a color is named after the portion of the spectrum that it absorbs, whereas for light a color is defined based on the portion of the spectrum that it emits.
- Consequently, *mixing all three primary colors of light results in white* (i.e., the entire spectrum of visible light), whereas *mixing all three primary colors of paints results in black* (i.e., all colors have been absorbed, and nothing remains to reflect the incoming light).

11.1.1 BASIC CONCEPTS

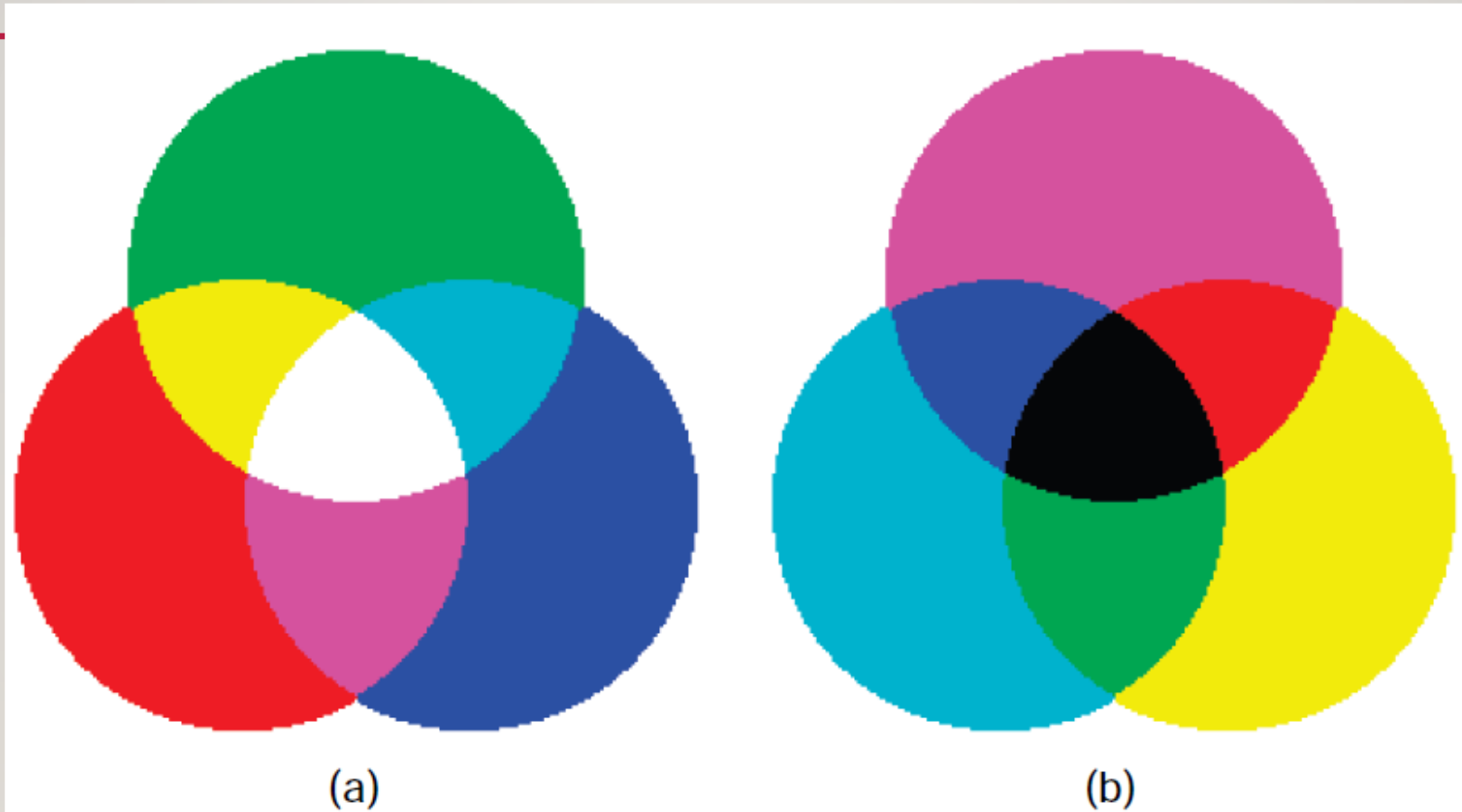


FIGURE 16.2 Additive (a) and subtractive (b) color mixtures.

11.1.1 BASIC CONCEPTS

- The use of *the expression primary colors* to refer to red, green, and blue may lead to a common misinterpretation: that all visible colors can be obtained by mixing different amounts of each primary color, which is not true.
- A related phenomenon of *color perception*, the existence of color metamers, may have contributed to this confusion. *Color metamers* are combinations of primary colors (e.g., red and green) perceived by the HVS as another color (in this case, yellow) that could have been produced by a spectral color of fixed wavelength (of ≈ 580 nm).

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

- In color matching experiments performed in the late 1920s, subjects were asked to adjust *the amount of red, green, and blue on one patch* that were needed to match a color on a second patch. The results of such experiments are summarized in Figure 11.3.
- *The existence of negative values of red and green in this figure means that the second patch should be made brighter (i.e., equal amounts of red, green, and blue has to be added to the color) for the subjects to report a perfect match. Since adding amounts of primary colors on the second patch corresponds to subtracting them in the first, negative values can occur.*

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

- The amounts of three primary colors in a three-component additive color model (on the first patch) needed to match a test color (on the second patch) are called *tristimulus values*.

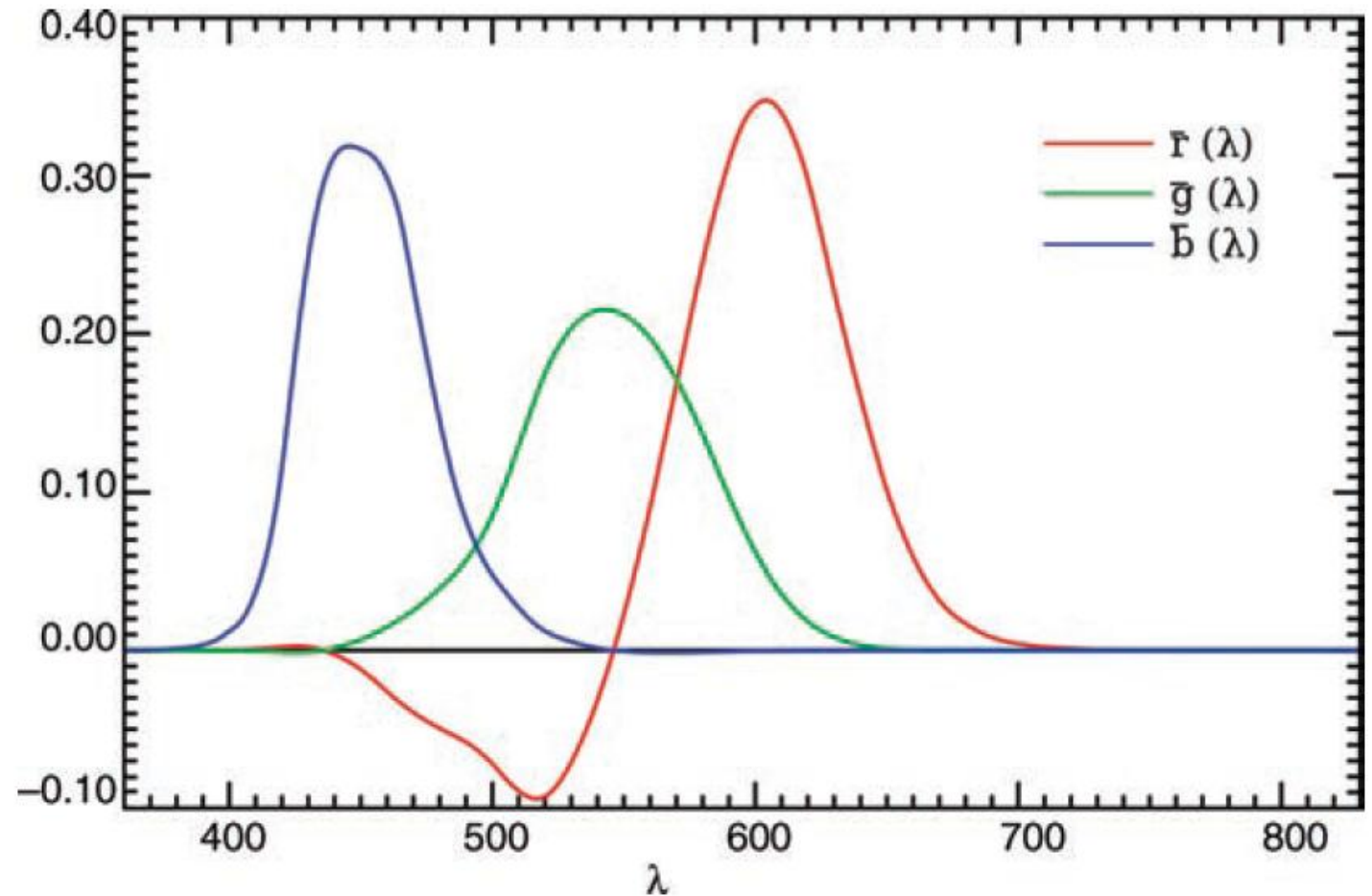


FIGURE 16.3 RGB color matching function (CIE 1931). Courtesy of Wikimedia Commons.

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

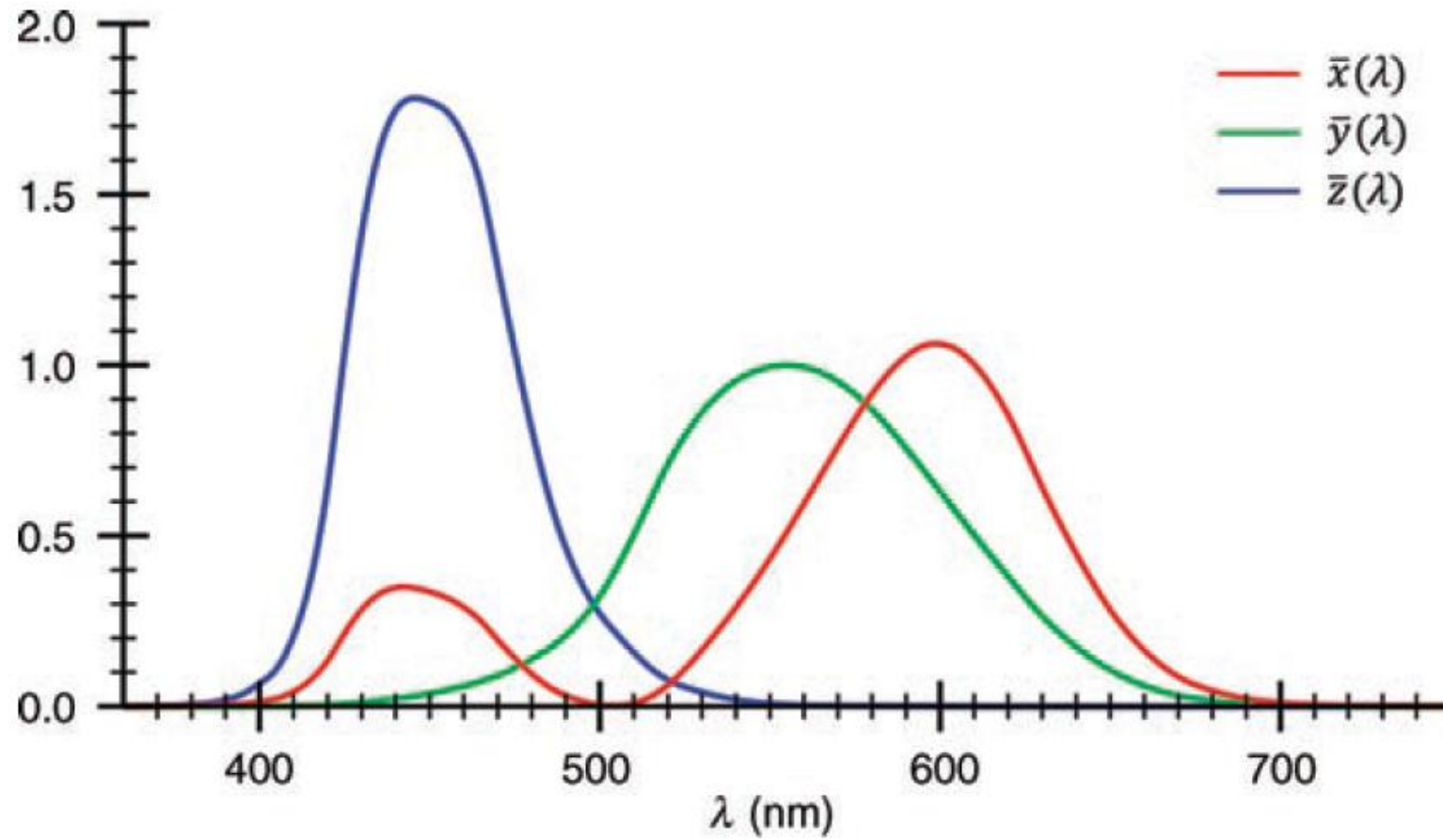


FIGURE 16.4 XYZ color matching function (CIE 1931). Courtesy of Wikimedia Commons.

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

- This model, whose color matching functions are shown in Figure 11.4, is known as the CIE XYZ (or CIE 1931) model. The tristimulus values of X, Y, and Z in Figure 11.4 are related to the values of R, G, and B in Figure 11.3 by the following linear transformations:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.431 & 0.342 & 0.178 \\ 0.222 & 0.707 & 0.071 \\ 0.020 & 0.130 & 0.939 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

and

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 3.063 & -1.393 & -0.476 \\ -0.969 & 1.876 & 0.042 \\ 0.068 & -0.229 & 1.069 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

- The CIE XYZ color space was designed so that the Y parameter corresponds to a measure of the brightness of a color. The chromaticity of a color is specified by two other parameters, x and y , known as chromaticity coordinates and calculated as

$$x = \frac{X}{X + Y + Z}$$

$$y = \frac{Y}{X + Y + Z}$$

where x and y are also called normalized tristimulus values. The third normalized tristimulus value, z , can just as easily be calculated as

$$z = \frac{Z}{X + Y + Z}$$

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

- The resulting CIE XYZ chromaticity diagram shows a horseshoe-shaped outer curved boundary, representing the spectral locus of wavelengths (in nm) along the visible light portion of the electromagnetic spectrum.
- The line of purples on a chromaticity diagram joins the two extreme points of the spectrum, suggesting that the sensation of purple cannot be produced by a single wavelength: *it requires a mixture of shortwave and longwave light and for this reason purple is referred to as a nonspectral color.* All colors of light are contained in the area in (x, y) bounded by the line of purples and the spectral locus, with pure white at its center.

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

- The inner triangle in Figure 11.6a represents a color gamut, that is, a range of colors that can be produced by a physical device, in this case a CRT monitor. Different color image display and printing devices and technologies exhibit *gamuts* of different shape and size, as shown in Figure 11.6.
- As a rule of thumb, the larger the gamut, the better the device's color reproduction capabilities.

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

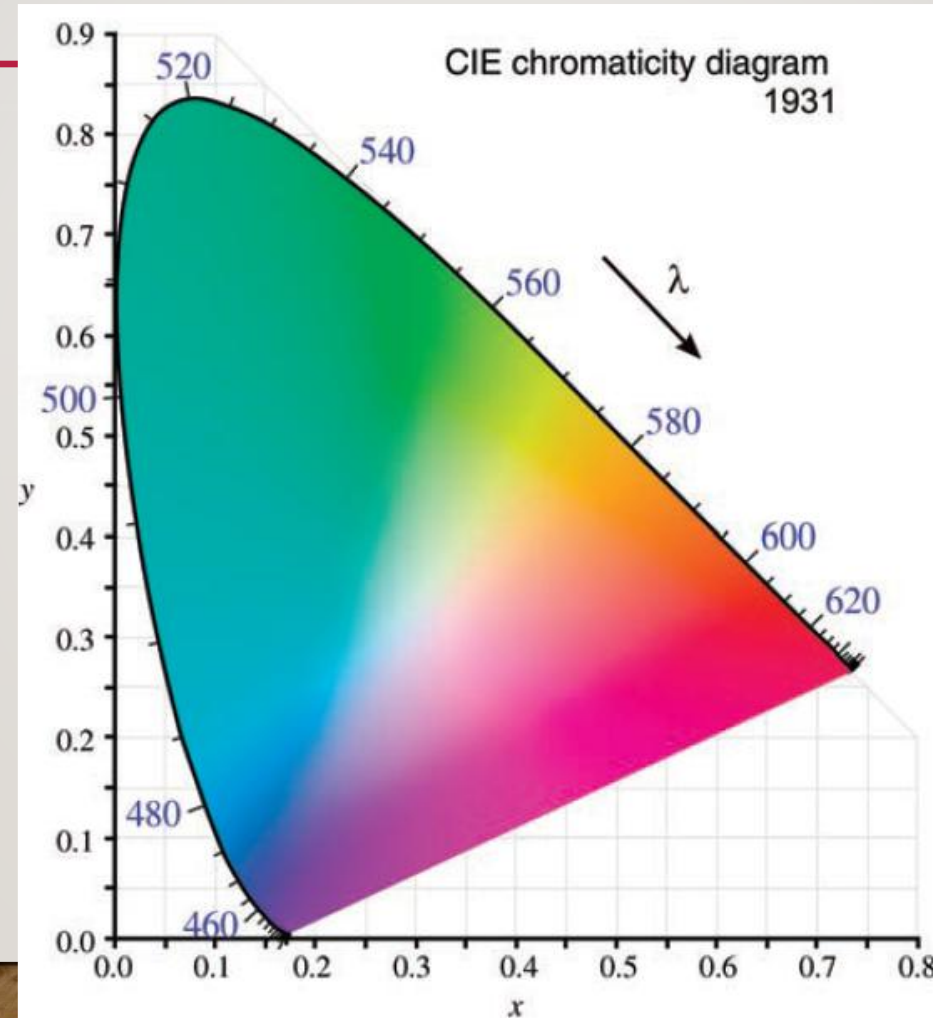


FIGURE 16.5 CIE XYZ color model.

11.1.2 THE CIE XYZ CHROMATICITY DIAGRAM

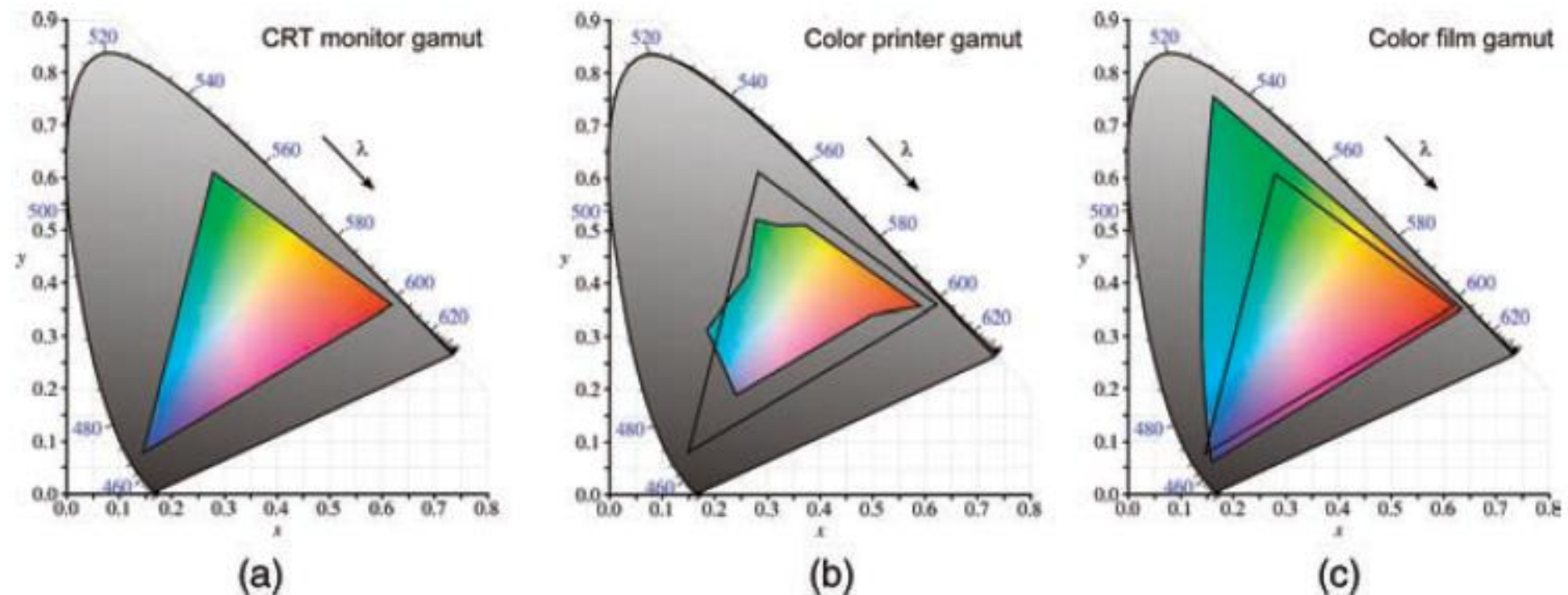


FIGURE 16.6 Color gamut for three different devices: (a) CRT monitor; (b) printer; (c) film. The RGB triangle is the same in all figures to serve as a reference for comparison.

11.1.3 PERCEPTUALLY UNIFORM COLOR SPACES

- One of the main limitations of the CIE XYZ chromaticity diagram lies in the fact that a distance on the xy plane *does not correspond to the degree of difference between two colors*. This was demonstrated in the early 1940s by David MacAdam, who conducted experiments asking subjects to report noticeable changes in color (relative to a starting color stimulus).
- The result of that study is illustrated in Figure 11.7, showing the resulting MacAdam ellipses: *regions on the chromaticity diagram corresponding to all colors that are indistinguishable, to the average human eye, from the color at the center of the ellipse*.

11.1.3 PERCEPTUALLY UNIFORM COLOR SPACES

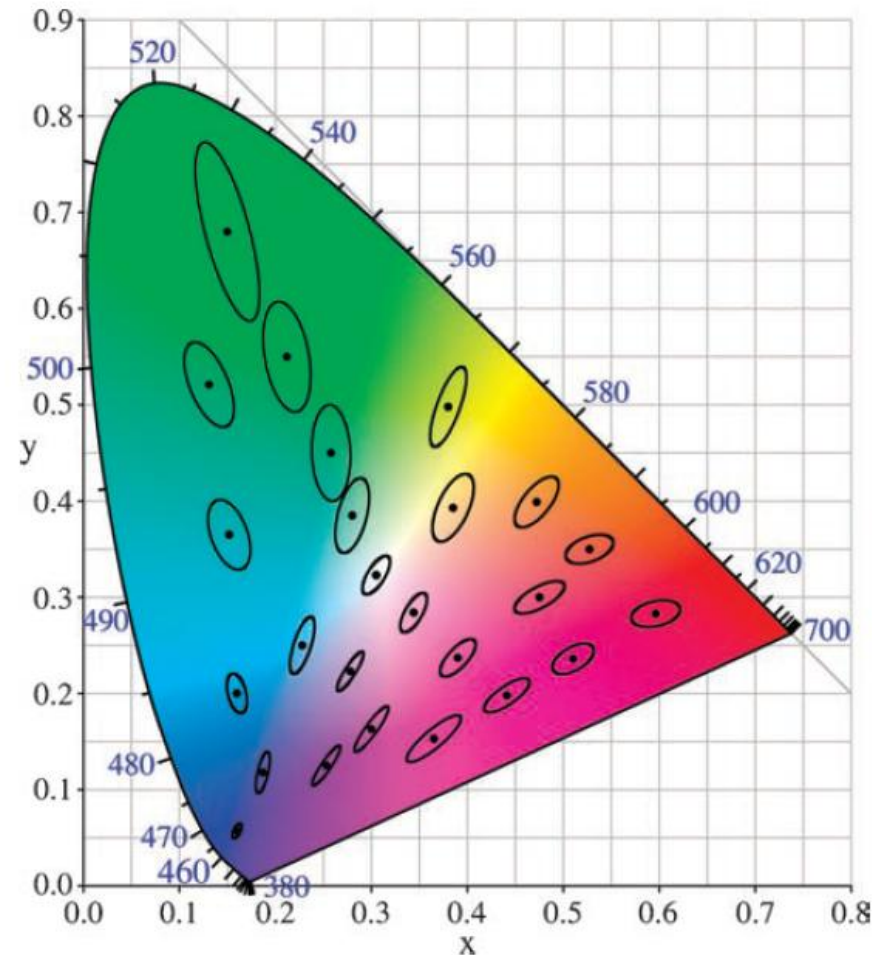


FIGURE 16.7 MacAdam ellipses overlapped on the CIE 1931 chromaticity diagram. Courtesy of Wikimedia Commons.

11.1.3 PERCEPTUALLY UNIFORM COLOR SPACES

In MATLAB

- The IPT has an extensive support for conversion among CIE color spaces. Converting from one color space to another is usually accomplished by using function *makecform* (to create a color transformation structure that defines the desired color space conversion) followed by *applycform*, which takes the color transformation structure as a parameter.



11.1.3 PERCEPTUALLY UNIFORM COLOR SPACES

TABLE 11.1 IPT Functions for CIE XYZ and CIELAB Color Spaces

Function	Description
xyz2double	Converts an $M \times 3$ or $M \times N \times 3$ array of XYZ color values to double
xyz2uint16	Converts an $M \times 3$ or $M \times N \times 3$ array of XYZ color values to uint16
lab2double	Converts an $M \times 3$ or $M \times N \times 3$ array of $L^*a^*b^*$ color values to double
lab2uint16	Converts an $M \times 3$ or $M \times N \times 3$ array of $L^*a^*b^*$ color values to uint16
lab2uint8	Converts an $M \times 3$ or $M \times N \times 3$ array of $L^*a^*b^*$ color values to uint8
whitepoint	Returns a 3×1 vector of XYZ values scaled so that $Y = 1$

11.1.4 ICC PROFILES

- An ICC (*International Color Consortium*) profile is a standardized description of a color input or output device, or a color space, according to standards established by the ICC. Profiles are used to define a mapping between the device source or target color space and a profile connection space (PCS), which is either CIELAB ($L^*a^*b^*$) or CIEXYZ.

In MATLAB

- The IPT has several functions to support ICC profile operations. They are listed in Table 11.2.

11.1.4 ICC PROFILES

TABLE 11.2 IPT Functions for ICC Profile Manipulation

Function	Description
iccread	Reads an ICC profile into the MATLAB workspace
iccfind	Finds ICC color profiles on a system, or a particular ICC color profile whose description contains a certain text string
iccroot	Returns the name of the directory that is the default system repository for ICC profiles
iccwrite	Writes an ICC color profile to disk file

11.2 COLOR MODELS

- A color model (also called color space or color system) is a specification of a coordinate system and a subspace within that system where each color is represented by a single point.
- There have been many different color models proposed over the last 400 years. Contemporary color models have also evolved to specify colors for different purposes (e.g., photography, physical measurements of light, color mixtures, etc.). In this section, we discuss the most popular color models used in image processing.



11.2.1 THE RGB COLOR MODEL

- **The RGB color model** is based on a *Cartesian coordinate system* whose axes represent the three primary colors of light (R, G, and B), usually normalized to the range $[0, 1]$ (Figure 16.8). *The eight vertices of the resulting cube correspond to the three primary colors of light, the three secondary colors, pure white, and pure black.* Table 16.3 shows the R, G, and B values for each of these eight vertices.
- RGB color coordinates are often represented in *hexadecimal notation*, with individual components varying from 00 (decimal 0) to FF (decimal 255). For example, a pure (100% saturated) red would be denoted *FF0000*, whereas a slightly desaturated yellow could be written as *CCCC33*.

11.2.1 THE RGB COLOR MODEL

- The number of discrete values of R, G, and B is a function of the pixel depth, defined as the number of bits used to represent each pixel: a typical value is 24 bits = 3 image planes x 8 bits per plane.
- The resulting cube—with more than 16 million possible color combinations.

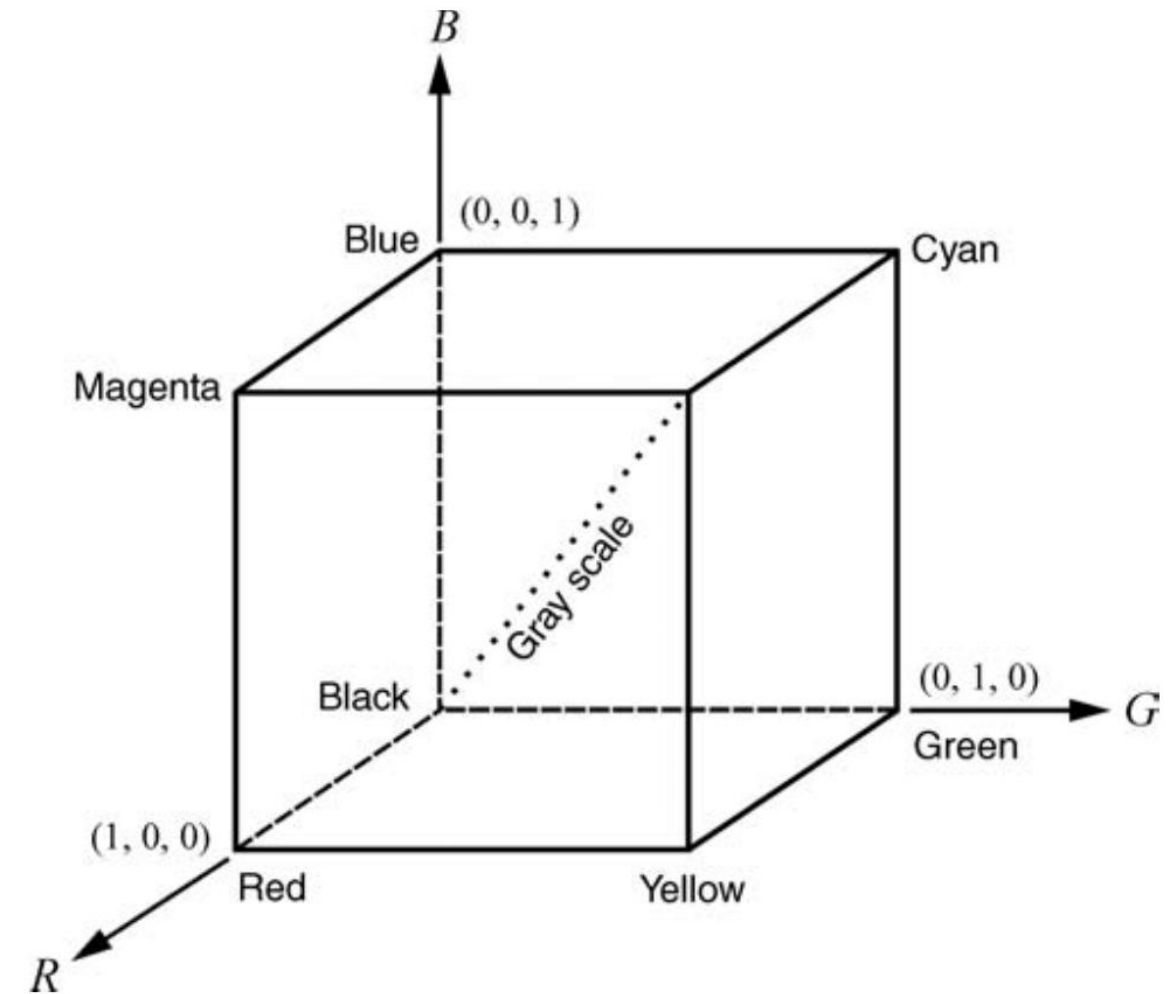


FIGURE 16.8 RGB color model.

11.2.1 THE RGB COLOR MODEL

TABLE 11.3 R, G, and B Values for Eight Representative Colors Corresponding to the Vertices of the RGB Cube

Color Name	R	G	B
Black	0	0	0
Blue	0	0	1
Green	0	1	0
Cyan	0	1	1
Red	1	0	0
Magenta	1	0	1
Yellow	1	1	0
White	1	1	1

11.2.1 THE RGB COLOR MODEL

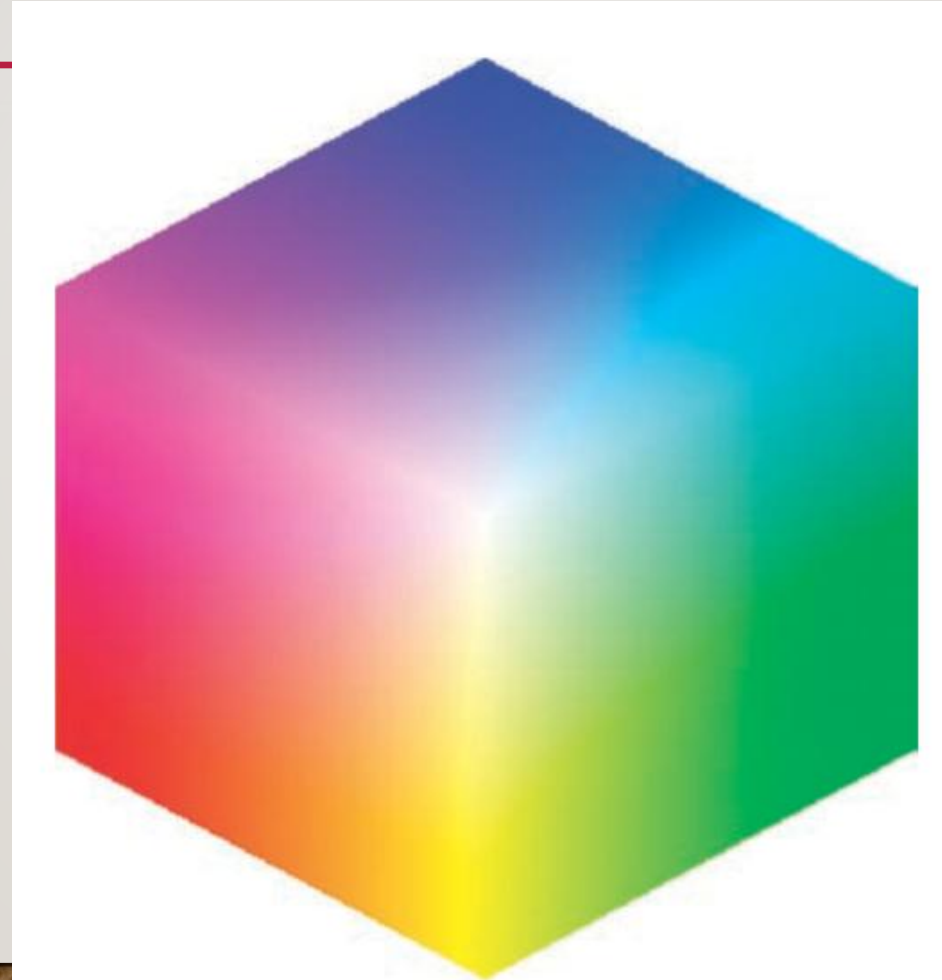


FIGURE 16.9 RGB color cube.

11.2.2 THE CMY AND CMYK COLOR MODELS

- The CMY model is based on the three primary colors of pigments (cyan, magenta, and yellow). It is used for color printers, where each primary color usually corresponds to an ink (or toner) cartridge. Since the addition of equal amounts of each primary to produce black usually produces unacceptable, muddy looking black, in practice, a fourth color, blackK, is added, and the resulting model is called CMYK.
- The conversion from RGB to CMY is straightforward:

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

- The inverse operation (conversion from CMY to RGB), although it is equally easy from a mathematical standpoint, is of little practical use.

11.2.3 THE HSV COLOR MODEL

- **Color models** such as the RGB and CMYK described previously are very convenient to specify color coordinates for display or printing, respectively. They are not, however, useful to capture a typical human description of color. After all, none of us goes to a store looking for a FFFFCC shirt to go with the FFCC33 jacket we got for our birthday.
- Rather, the human perception of color is best described in terms of *hue*, *saturation*, and *lightness*. *Hue* describes the color type, or tone, of the color (and very often is expressed by the “color name”), *saturation* provides a measure of its purity (or how much it has been diluted in white), and *lightness* refers to the intensity of light reflected from objects.

11.2.3 THE HSV COLOR MODEL

- For representing colors in a way that is closer to *the human description*, a family of color models have been proposed. The common aspect among these models is their ability to dissociate the dimension of intensity (also called brightness or value) from the chromaticity—expressed as a combination of *hue* and *saturation*—of a color.



11.2.3 THE HSV COLOR MODEL

- **The HSV** (sometimes called *HSB*) color model can be obtained by *looking at the RGB color cube along its main diagonal* (or gray axis), which results in a hexagon-shaped color palette. As we move along the main axis in the pyramid in Figure 11.10, the hexagon gets smaller, *corresponding to decreasing values of V , from 1 (white) to 0 (black)*.
- For any hexagon, the three primary and the three secondary colors of light are represented in its vertices. *Hue*, therefore, is specified as an angle relative to the origin (the red axis by convention). Finally, saturation is specified by the distance to the axis: *the longer the distance, the more saturated the color*.

11.2.3 THE HSV COLOR MODEL

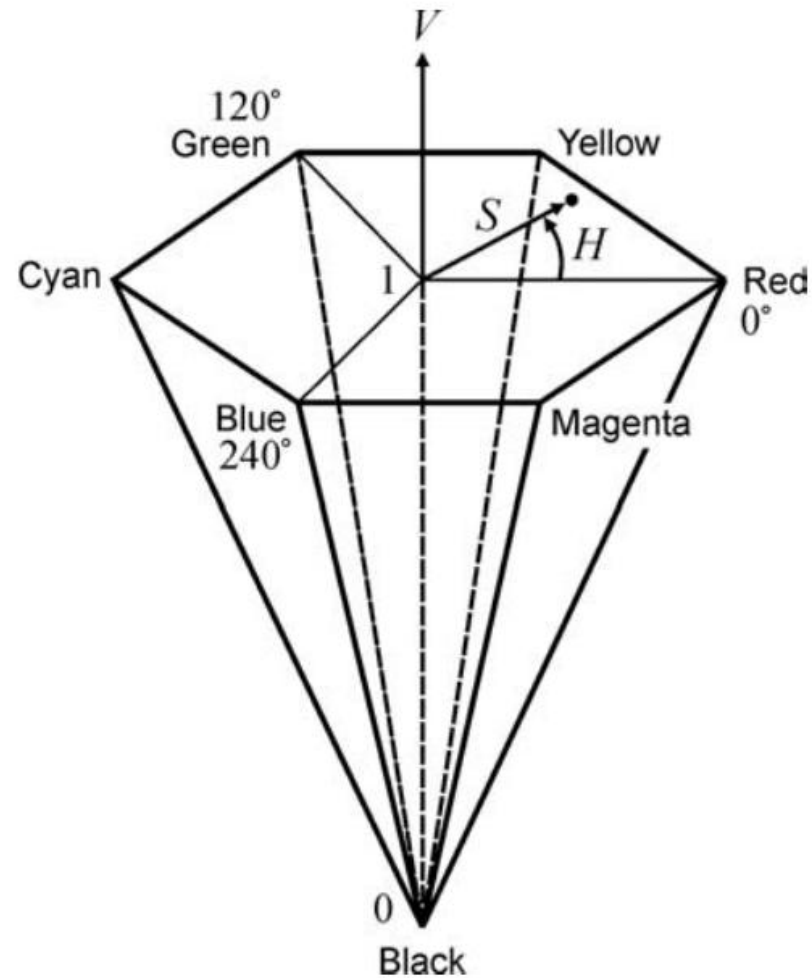
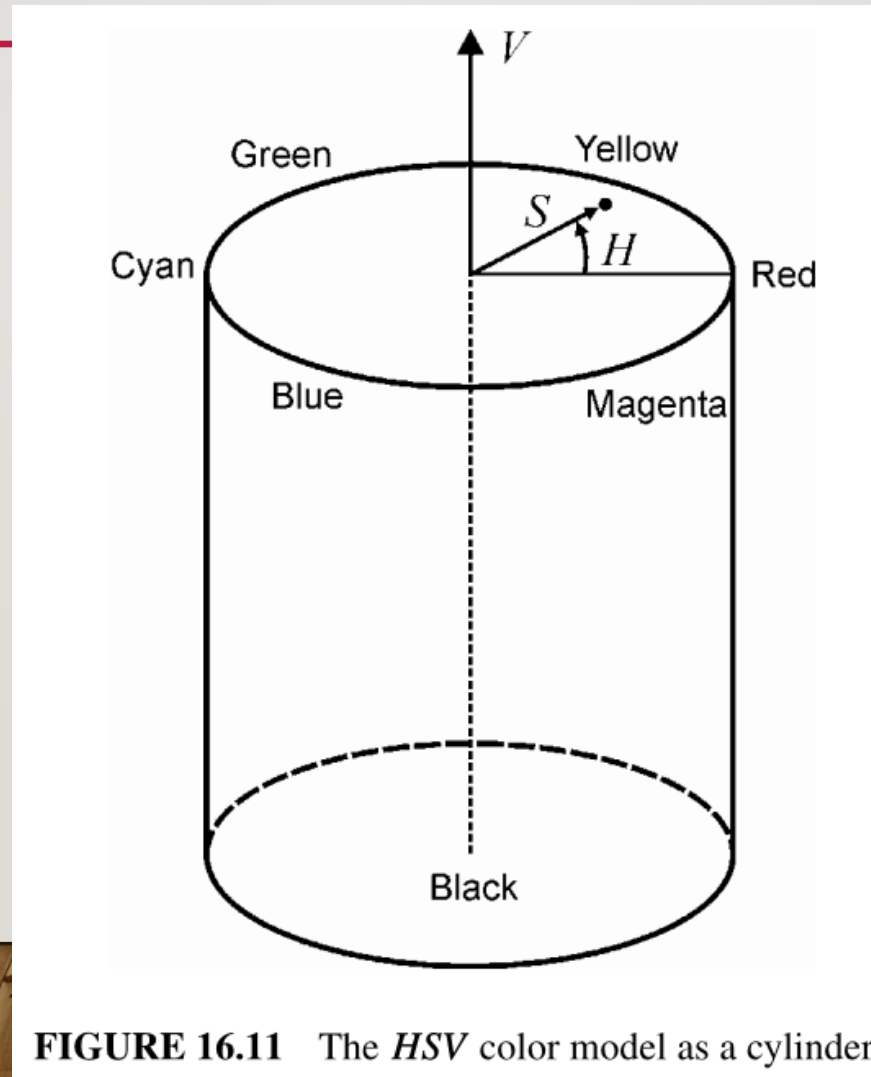


FIGURE 16.10 The HSV color model as a hexagonal cone.

11.2.3 THE HSV COLOR MODEL

- Figure 11.11 shows an alternative representation of the HSV color model in which the hexcone is replaced by a cylinder. Figure 16.12 shows yet another equivalent three-dimensional representation for the HSV color model, as a cone with circular-shaped base.
- In summary, the main advantages of the HSV color model (and its closely related alternatives) are its ability to match the human way of describing colors and to allow for independent control over hue, saturation, and intensity (value). The ability to isolate the intensity component from the other two—which are often collectively called chromaticity components—is a requirement in many color image processing algorithms.

11.2.3 THE HSV COLOR MODEL



11.2.3 THE HSV COLOR MODEL

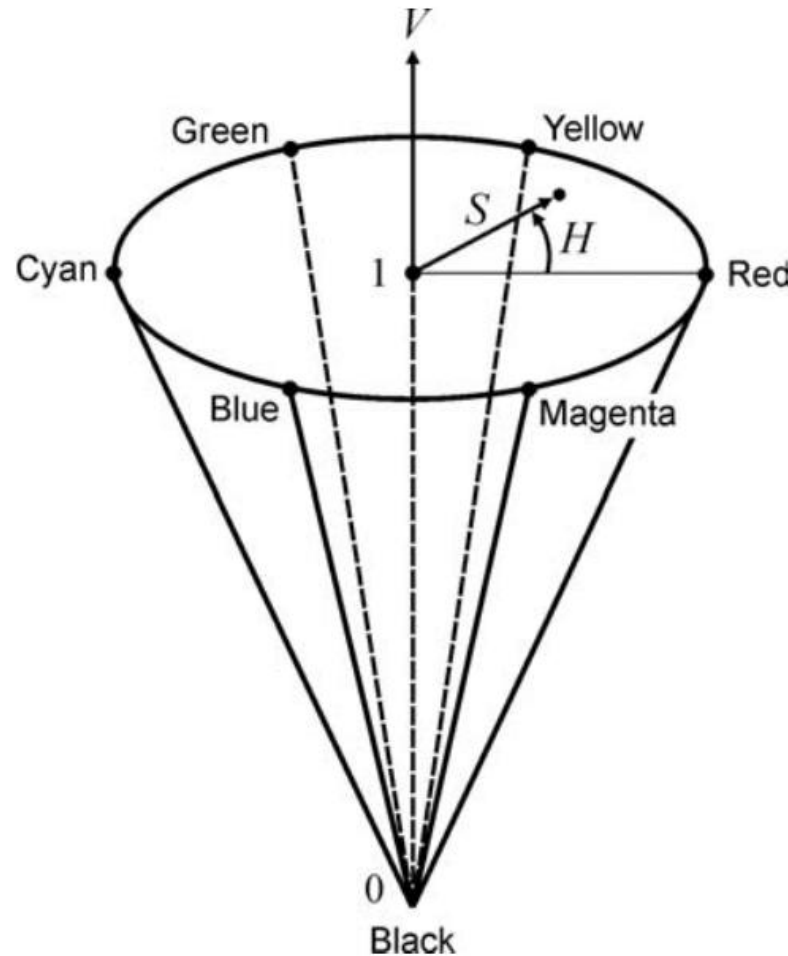


FIGURE 16.12 The HSV color model as a cone.

11.2.4 THE YIQ (NTSC) COLOR MODEL

- The NTSC color model is used in the American standard for analog television, which will be described in more detail in Chapter 20. One of the main advantages of this model is the ability to separate grayscale contents from color data, a major design requirement at a time when emerging color TV sets and transmission equipment had to be backward compatible with their B&W predecessors. In the NTSC color model, the three components are luminance (Y) and two color-difference signals hue (I) and saturation (Q).
- Conversion from RGB to YIQ can be performed using the transformation

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.274 & -0.322 \\ 0.211 & -0.523 & 0.312 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

11.2.5 THE YCBCR COLOR MODEL

- The **YCbCr color model** is *the most popular color representation for digital video*. In this format, one component represents luminance (Y), while the other two are color-difference signals: Cb (the difference between the blue component and a reference value) and Cr (the difference between the red component and a reference value).
- Conversion from RGB to YCbCr is possible using the transformation

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.331 & 0.500 \\ 0.500 & -0.419 & -0.081 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

11.3 REPRESENTATION OF COLOR IMAGES IN MATLAB

- Color images are usually represented as RGB (24 bits per pixel) or indexed with a palette (color map), usually of size 256. These representation modes are independent of file format (although GIF images are usually indexed, and JPEG images typically are not). In this section, we provide a more detailed analysis of color image representation in MATLAB.

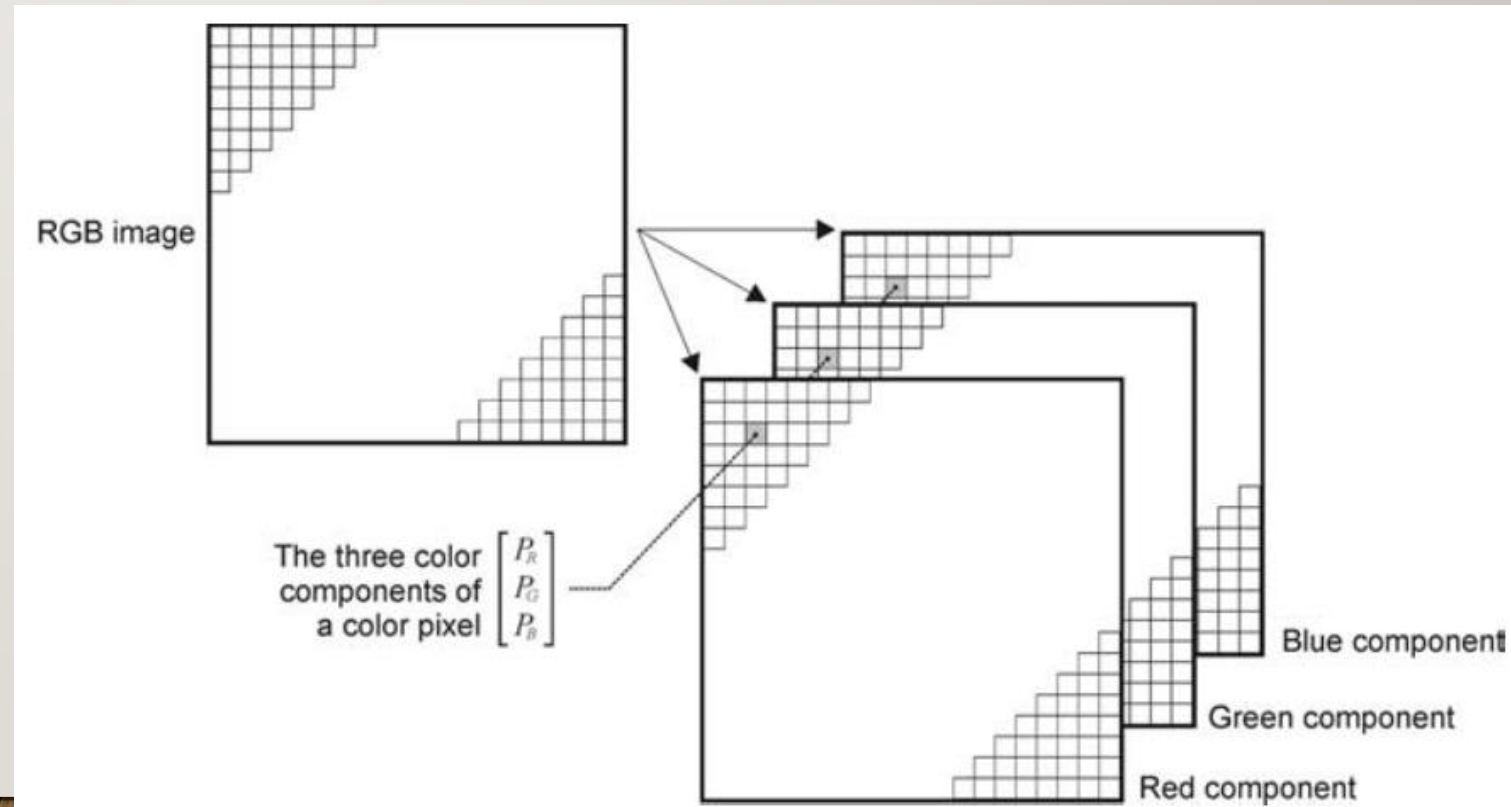


FIGURE 16.13 RGB color image representation.

11.3.1 RGB IMAGES

- An RGB color image in MATLAB corresponds to a 3D array of dimensions $M \times N \times 3$, where M and N are the image's height and width (respectively) and 3 is the number of color planes (channels). Each color pixel is represented as a triple containing the values of its R, G, and B components (Figure 11.13). Each individual array of size $M \times N$ is called *a component image* and *corresponds to one of the color channels*: red, green, or blue. The data class of the component images determines their range of values.
- For RGB images of class *double*, the range of values is $[0.0, 1.0]$, whereas for classes *uint8* or *uint16*, the ranges are $[0, 255]$ and $[0, 65535]$, respectively. RGB images typically have *a bit depth of 24 bits per pixel* (8 bits per pixel per component image), resulting in a total number of $(2^8)^3 = 16,777,216$ colors.

EXAMPLE 11.1

- The following MATLAB sequence can be used to open, verify the size (in this case, $384 \times 512 \times 3$) and data class (in this case, `uint8`), and display an RGB color image (Figure 11.14).

```
I = imread('peppers.png');  
size(I)  
class(I)  
subplot(2,2,1), imshow(I), title('Color image (RGB)');  
subplot(2,2,2), imshow(I(:,:,1)), title('Red component');  
subplot(2,2,3), imshow(I(:,:,2)), title('Green component');  
subplot(2,2,4), imshow(I(:,:,3)), title('Blue component');
```

EXAMPLE 11.1

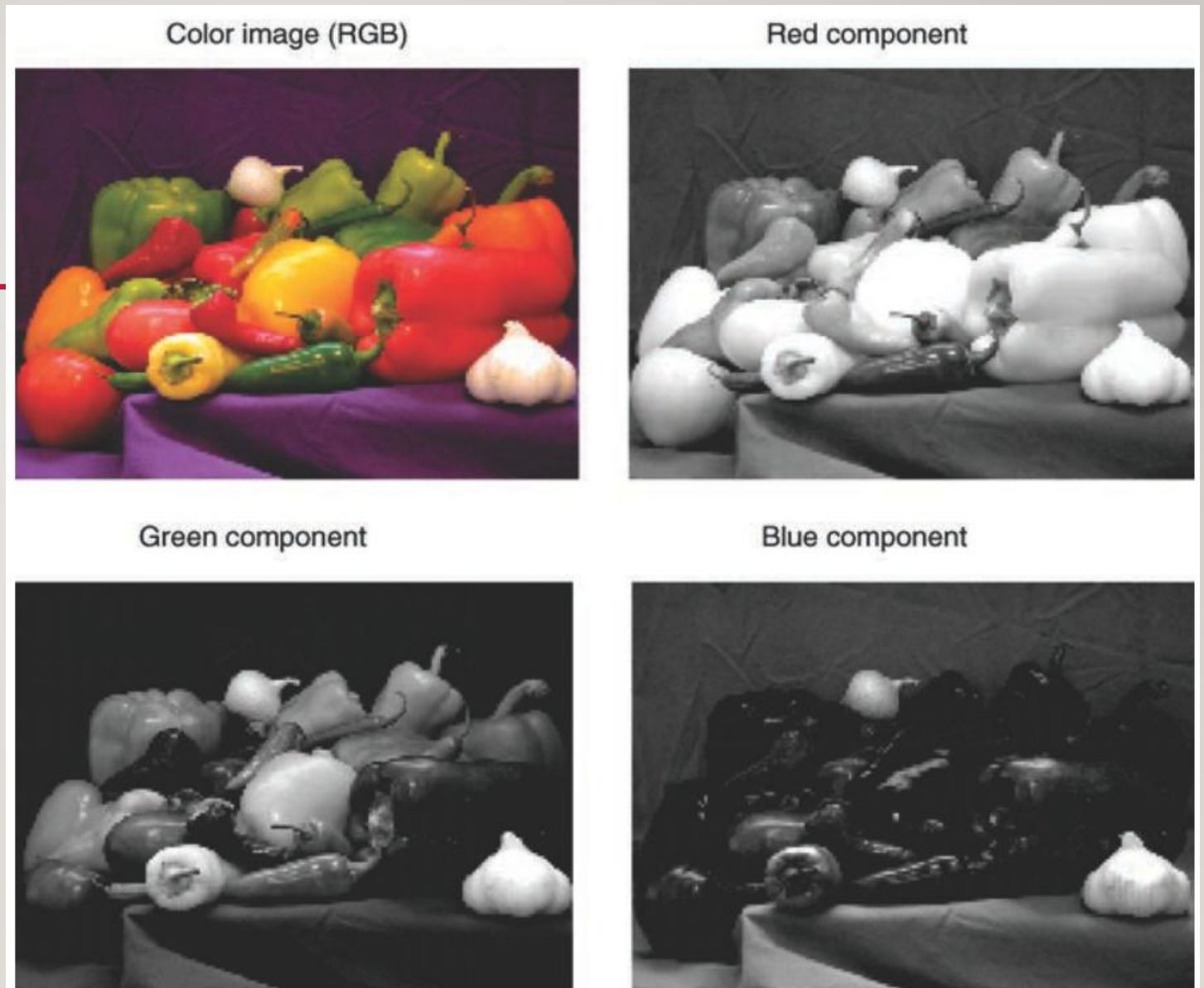


FIGURE 16.14 *RGB* image and its three color components (or *channels*). Original image: courtesy of MathWorks.

11.3.2 INDEXED IMAGES

- An **indexed image** is a matrix of integers (X), where each integer refers to a particular row of RGB values in a secondary matrix (*map*) known as a **color map**. The image can be represented by an array of class `uint8`, `uint16`, or `double`. The color map array is an $M \times 3$ matrix of class `double`, where each element's value is within the range $[0.0, 1.0]$. Each row in the color map represents R (red), G (green), and B (blue) values, in that order.
- The indexing mechanism works as follows (Figure 11.15): if X is of class `uint8` or `uint16`, all components with value 0 point to the first row in *map*, all components with value 1 point to the second row, and so on. If X is of class `double`, all components with value less than or equal to 1.0 point to the first row and so on.

11.3.2 INDEXED IMAGES

- MATLAB has many built-in color maps (which can be accessed using the function `colormap`), briefly described in Table 11.4. In addition, you can easily create your own color map by defining an array of class `double` and size $M \times 3$, where each element is a floating-point value in the range $[0.0, 1.0]$.

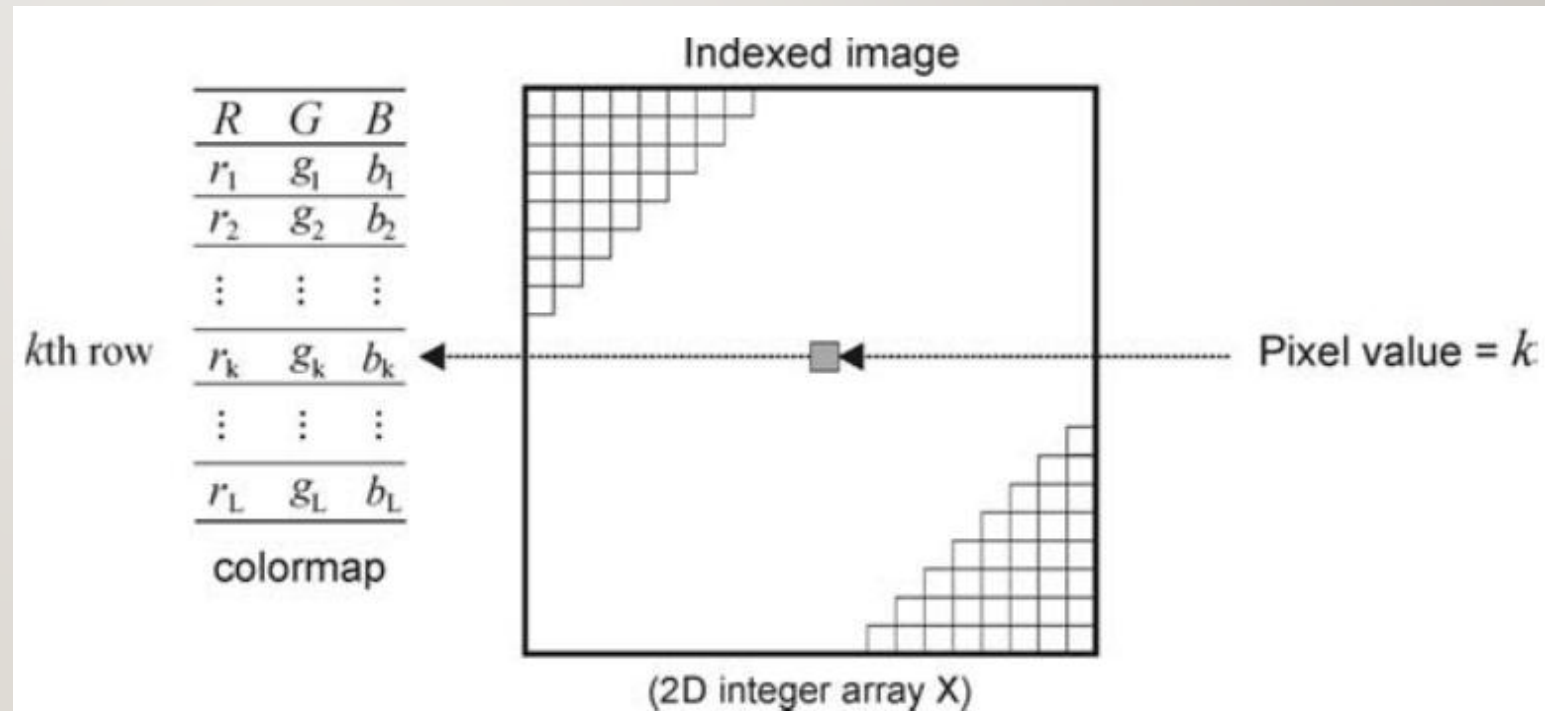


FIGURE 16.15 Indexed color image representation.

11.3.2 INDEXED IMAGES

TABLE 11.4 Color Maps in MATLAB

Name	Description	Name	Description
hsv	Hue–saturation–value color map	colorcube	Enhanced color-cube color map
hot	Black–red–yellow–white color map	vga	Windows color map for 16 colors
gray	Linear gray-scale color map	jet	Variant of HSV
bone	Gray scale with tinge of blue color map	prism	Prism color map
copper	Linear copper-tone color map	cool	Shades of cyan and magenta color map
pink	Pastel shades of pink color map	autumn	Shades of red and yellow color map
white	All white color map	spring	Shades of magenta and yellow color map
flag	Alternating red, white, blue, and black color map	winter	Shades of blue and green color map
lines	Color map with the line colors	summer	Shades of green and yellow color map

EXAMPLE 11.2

- The following MATLAB sequence can be used to load a built-in indexed image, verify its size (in this case, 200 x 300) and data class (in this case, *double*), verify the color map's size (in this case, 81 x 3) and data class (in this case, *double*), and display the image (Figure 11.16).

```
load clown
```

```
size(X)
```

```
class(X)
```

```
size(map)
```

```
class(map)
```

```
imshow(X,map), title('Color (Indexed)')
```

EXAMPLE 11.2

Color(indexed)



FIGURE 16.16 A built-in indexed image. Original image: courtesy of MathWorks.

11.3.2 INDEXED IMAGES

- MATLAB has many useful functions for manipulating indexed color images:
 - If we need to approximate an indexed image by one with fewer colors, we can use MATLAB's `imapprox` function.
 - Conversion between RGB and indexed color images is straightforward, thanks to functions `rgb2ind` and `ind2rgb`.
 - Conversion from either color format to their grayscale equivalent is equally easy, using functions `rgb2gray` and `ind2gray`.
 - We can also create an index image from an RGB image by dithering the original image using function `dither`.
 - Function `grayslice` creates an indexed image from an intensity (grayscale) image by thresholding and can be used in pseudocolor image processing.
 - Function `gray2ind` converts an intensity (grayscale) image into its indexed image equivalent. It is different from `grayslice`. In this case, the resulting image is monochrome, just as the original one; only the internal data representation has changed.

11.4 PSEUDOCOLOR IMAGE PROCESSING

- The purpose of pseudocolor image processing techniques is to enhance a monochrome image for human viewing purposes. Their rationale is that subtle variations of gray levels may very often mask or hide regions of interest within an image. This can be particularly damaging if the masked region is relevant to the application domain (e.g., the presence of a tumor in a medical image).
- Since the human eye is capable of discerning thousands of color hues and intensities, compared to only less than 100 shades of gray, replacing gray levels with colors leads to better visualization and enhanced capability for detecting relevant details within the image.

11.4 PSEUDOCOLOR IMAGE PROCESSING

- The typical solution consists of using a color lookup table (LUT) designed to map the entire range of (typically 256) gray levels to a (usually much smaller) number of colors. For better results, contrasting colors should appear in consecutive rows in the LUT.
- The term *pseudocolor* is used to emphasize the fact that the assigned colors usually have no correspondence whatsoever with the true colors that might have been present in the original image.



11.4.1 INTENSITY SLICING

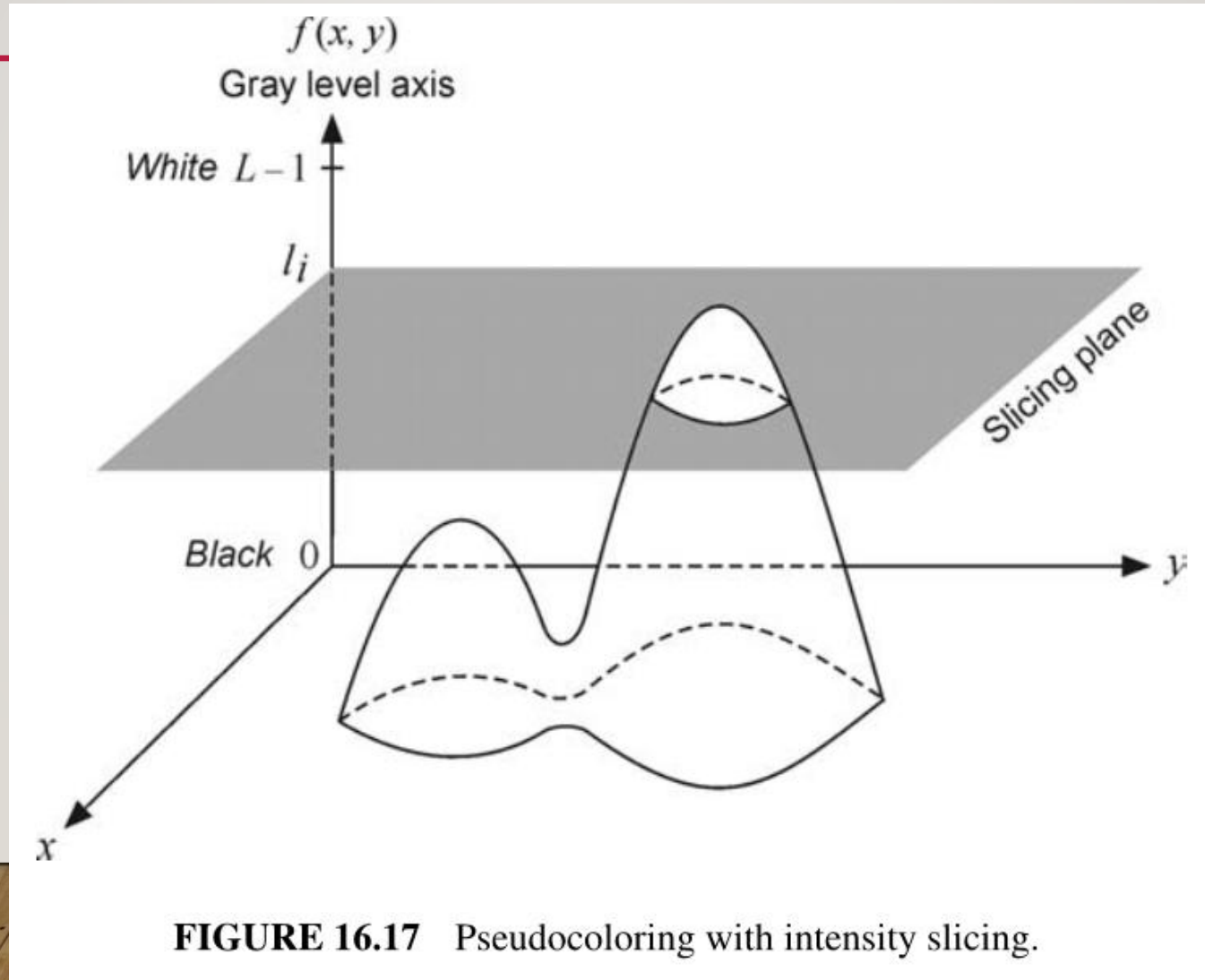
- The technique of intensity (or density) slicing is the simplest and best-known pseudocoloring technique.
- If we look at a monochrome image as if it were a 3D plot of gray levels versus spatial coordinates, where the most prominent peaks correspond to the brightest pixels, the technique corresponds to *placing several planes parallel to the coordinate plane of the image* (also known as the xy plane).



11.4.1 INTENSITY SLICING

- Each plane “slices” the 3D function in the area of intersection, resulting in several gray-level intervals. Each side of the plane is then assigned a different color.
- Figure 11.17 shows an example of intensity slicing using only one slicing plane at $f(x, y) = l_i$ and Figure 11.18 shows an alternative representation, where the chosen colors are indicated as c_1, c_2, c_3 , and c_4 . The idea can easily be extended to M planes and $M + 1$ intervals.

11.4.1 INTENSITY SLICING



11.4.1 INTENSITY SLICING

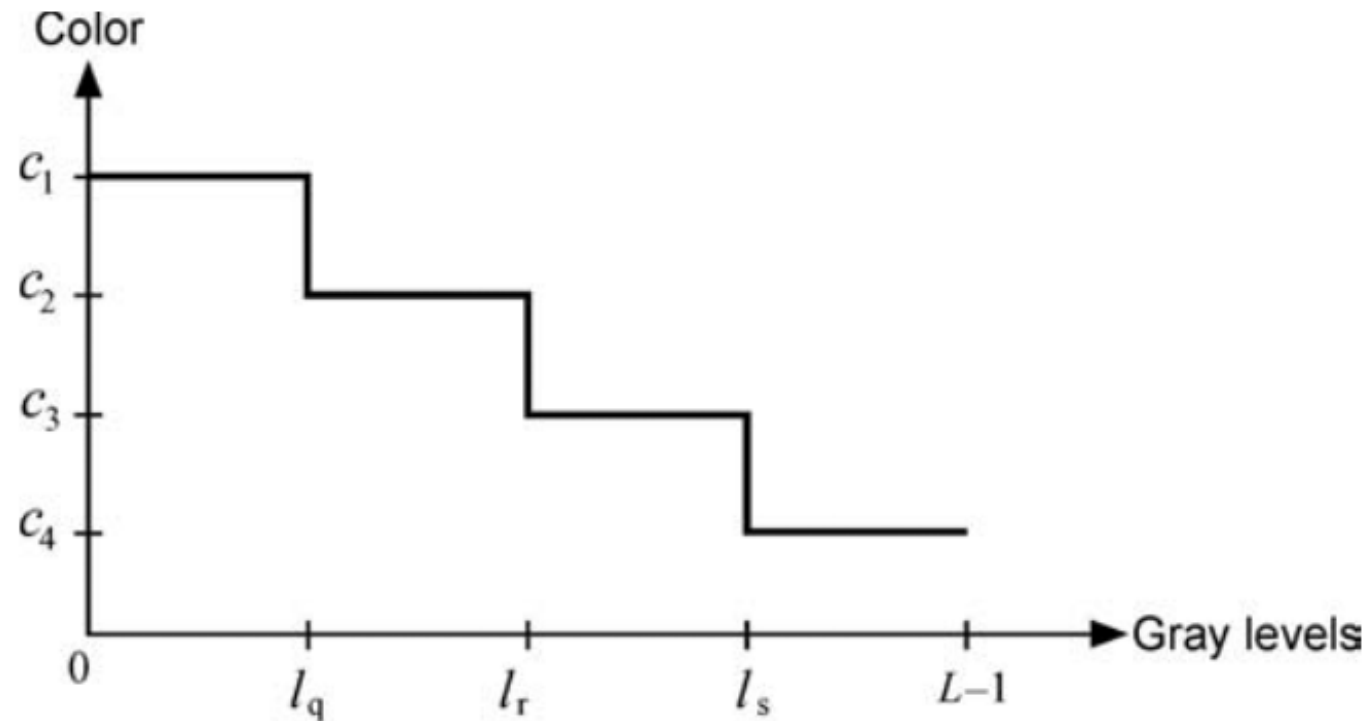


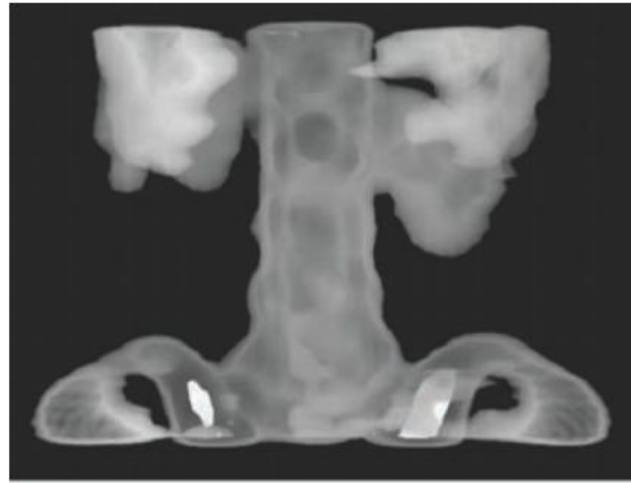
FIGURE 16.18 An alternative representation of the intensity slicing technique for an image with L gray levels pseudocolored using four colors.

EXAMPLE 11.3

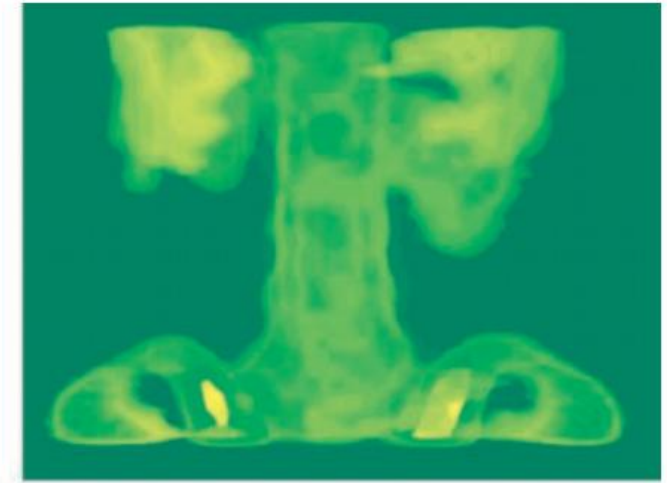
In MATLAB

- Intensity slicing can be accomplished using the *grayslice* function.
- Figure 11.19 shows an example of pseudocoloring with intensity slicing using 16 levels, the same input image (a), and three different color maps summer (b), hsv (c), and jet (d).

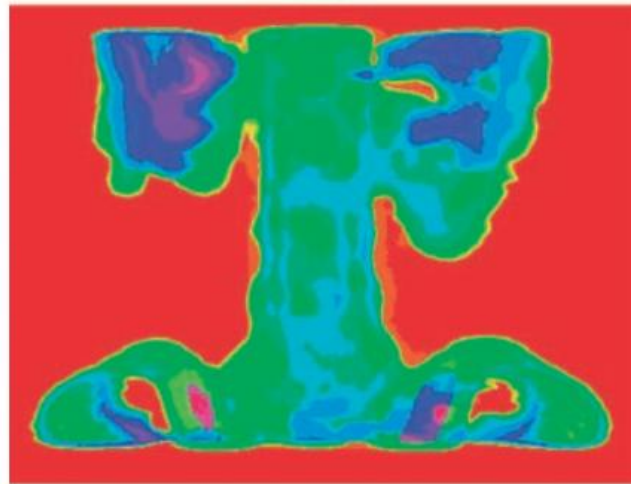
EXAMPLE 11.3



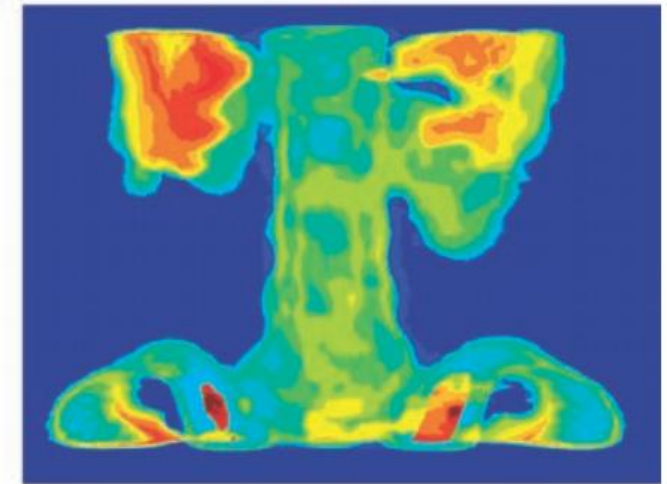
(a)



(b)



(c)



(d)

FIGURE 16.19 Pseudocoloring using intensity slicing: original image (a) and results of pseudocoloring using different color maps (b–d). Original image: courtesy of MathWorks.

11.6 TUTORIAL: PSEUDOCOLOR IMAGE PROCESSING

Goal

- The goal of this tutorial is to learn how to display *grayscale* images using *pseudocolors* in MATLAB.

Objectives

- *Learn how to use the `grayslice` function to perform intensity slicing.*
- *Learn how to specify color maps with a custom number of colors.*



11.6 TUTORIAL: PSEUDOCOLOR IMAGE PROCESSING

Procedure

We will start by exploring the *grayscale* function on a gradient image.

1. Create and display a gradient image.

```
I = repmat(uint8([0:255]),256,1);  
figure, subplot(1,2,1), subimage(I), title('Original Image');
```

2. Slice the image and display the results.

```
I2 = grayscale(I,16);  
subplot(1,2,2), subimage(I2,colormap(winter(16))), ...  
title('Pseudo-colored with "winter" colormap')
```

Question 1. Why did we use the *subimage* function to display the images (instead of the familiar *imshow*)?

11.6 TUTORIAL: PSEUDOCOLOR IMAGE PROCESSING

Question 2. What does the value 16 represent in the function call for `grayscale`?

Question 3. In the statement `subimage(I2,colormap(winter(16)))`, what does the value 16 represent?

In the above procedure, we sliced the image into equal partitions—this is the default for the `grayscale` function. We will now learn how to slice the range of grayscale values into unequal partitions.

3. Slice the image into unequal partitions and display the result.

```
levels = [0.25*255, 0.75*255, 0.9*255];
```

```
I3 = grayscale(I,levels);
```

```
figure, imshow(I3,spring(4))
```


11.6 TUTORIAL: PSEUDOCOLOR IMAGE PROCESSING

Question 4. The original image consists of values in the range $[0, 255]$. If our original image values ranged $[0.0, 1.0]$, how would the above code change?

Now that we have seen how pseudocoloring works, let us apply it to an image where this visual information might be useful.

4. Clear all variables and close any open figures.

5. Load and display the mri.jpg.

```
I = imread('mri.jpg');
```

```
figure, subplot(1,2,1), subimage(I), title('Original Image');
```

11.6 TUTORIAL: PSEUDOCOLOR IMAGE PROCESSING

6. Pseudocolor the image.

```
I2 = grayslice(I, 16);  
subplot(1,2,2), subimage(I2, colormap(jet(16))), ...  
title('Pseudo-colored with "jet" colormap');
```

Question 5. In the previous steps, we have specified how many colors we want in our color map. If we do not specify this number, how does MATLAB determine how many colors to return in the color map?

11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

Goal

- The goal of this tutorial is to learn how to convert between color spaces and perform filtering on color images in MATLAB.

Objectives

- *Learn how to convert from RGB to HSV color space using the `rgb2hsv` function.*
- *Learn how to convert from HSV to RGB color space using the `hsv2rgb` function.*
- *Explore smoothing and sharpening in the RGB and HSV color spaces.*



11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

Procedure

We will start by exploring the `rgb2hsv` function.

1. Load the `onions.png` image and display its RGB components.

```
I = imread('onion.png');  
figure, subplot(2,4,1), imshow(I), title('Original Image');  
subplot(2,4,2), imshow(I(:,:,1)), title('R component');  
subplot(2,4,3), imshow(I(:,:,2)), title('G component');  
subplot(2,4,4), imshow(I(:,:,3)), title('B component');
```

2. Convert the image to HSV and display its components.

```
lhsv = rgb2hsv(I);  
subplot(2,4,6), imshow(lhsv(:,:,1)), title('Hue');  
subplot(2,4,7), imshow(lhsv(:,:,2)), title('Saturation');  
subplot(2,4,8), imshow(lhsv(:,:,3)), title('Value');
```


11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

Question 1. Why do we not display the HSV equivalent of the image?

When viewing the components of an RGB image, the grayscale visualization of each component is intuitive because the intensity within that component corresponds to how much of the component is being used to generate the final color. Visualization of the components of an HSV image is not as intuitive. You may have noticed that when displaying the hue, saturation, and value components, hue and saturation do not give you much insight as to what the actual color is. The value component, on the other hand, appears to be a grayscale version of the image.



11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

3. Convert the original image to grayscale and compare it with the value component of the HSV image.

```
lgray = rgb2gray(I);
```

```
figure, subplot(1,2,1), imshow(lgray), title('Grayscale');
```

```
subplot(1,2,2), imshow(lhsv(:,:,3)), title('Value component');
```

Question 2. How does the grayscale version of the original image and the value component of the HSV image compare?

Procedures for filtering a color image will vary depending on the color space being used. Let us first learn how to apply a smoothing filter on an RGB image.

11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

4. Apply a smoothing filter to each component and then reconstruct the image.

```
fn = fspecial('average');  
I2r = imfilter(I(:,:,1), fn);  
I2g = imfilter(I(:,:,2), fn);  
I2b = imfilter(I(:,:,3), fn);  
I2(:,:,1) = I2r;  
I2(:,:,2) = I2g;  
I2(:,:,3) = I2b;  
figure, subplot(1,2,1), imshow(I), title('Original Image');  
subplot(1,2,2), imshow(I2), title('Averaged Image');
```

11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

Question 3. Do the results confirm that the RGB equivalent of averaging a grayscale image is to average each component of the RGB image individually?

Now let us see what happens when we perform the same operations on an HSV image. Note that in these steps, we will use the `hsv2rgb` function to convert the HSV image back to RGB so that it can be displayed.

5. Filter all components of the HSV image.

```
lHSV2h = imfilter(lHSV(:,:,1), fn);  
lHSV2s = imfilter(lHSV(:,:,2), fn);  
lHSV2v = imfilter(lHSV(:,:,3), fn);  
lHSV2(:,:,1) = lHSV2h;  
lHSV2(:,:,2) = lHSV2s;  
lHSV2(:,:,3) = lHSV2v;
```


11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

6. Display the results.

```
figure, subplot(2,3,1), imshow(lhsv(:,:,1)), ...  
title('Original Hue');  
subplot(2,3,2), imshow(lhsv(:,:,2)), ...  
title('Original Saturation');  
subplot(2,3,3), imshow(lhsv(:,:,3)), ...  
title('Original Value');  
subplot(2,3,4), imshow(lhsv2(:,:,1)), ...  
title('Filtered Hue');
```

11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

```
subplot(2,3,5), imshow(lhsv2(:,:,2)), ...  
title('Filtered Saturation');  
subplot(2,3,6), imshow(lhsv2(:,:,3)), ...  
title('Filtered Value');  
figure, subplot(1,2,1), imshow(I), title('Original Image');  
subplot(1,2,2), imshow(hsv2rgb(lhsv2)), ...  
title('HSV with all components filtered');
```

11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

Question 4. Based on the results, does it make sense to say that the HSV equivalent of averaging a grayscale image is to average each component of the HSV image individually?

7. Filter only the value component and display the results.

```
lHSV3(:,:, [1 2]) = lHSV(:,:, [1 2]);
```

```
lHSV3(:,:, 3) = lHSV2v;
```

```
figure, subplot(1,2,1), imshow(I), title('Original Image');
```

```
subplot(1,2,2), imshow(hsv2rgb(lHSV3)), ...
```

```
title('HSV with only value component filtered');
```

11.7 TUTORIAL: FULL-COLOR IMAGE PROCESSING

Question 5. How does this result compare with the previous one?

We can sharpen an HSV image following a similar sequence of steps.

8. Sharpen the HSV image and display the result.

```
fn2 = fspecial('laplacian',0);  
lHSV4v = imfilter(lHSV(:,:,3), fn2);  
lHSV4(:,:, [1 2]) = lHSV(:,:, [1 2]);  
lHSV4(:,:,3) = imsubtract(lHSV(:,:,3), lHSV4v);  
figure, subplot(1,2,1), imshow(I), title('Original Image');  
subplot(1,2,2), imshow(hsv2rgb(lHSV4)), ...  
title('HSV sharpened');
```

Question 6. How would we perform the same sharpening technique on an RGB image?

WHAT HAVE WE LEARNED?

- This chapter introduced the most important concepts and terms related to *color perception, representation, and processing*. Understand the meaning of the main terms in *colorimetry* is a required step toward the understanding of color image processing.
- There are many color models used to represent and quantify color information. Some of the most popular (and their use) models are as follows:
 - RGB, CMY(K): display and printing devices
 - YIQ, YCbCr: television and video
 - XYZ: color standardization
 - CIELAB, CIELUV: perceptual uniformity
 - HSV, HSI, HSL: intuitive description of color properties

WHAT HAVE WE LEARNED?

- **Color images** can be represented in MATLAB either as an $M \times N \times 3$ array (one per color channel) or as an $M \times N$ array of indices (pointers) to a secondary (usually 256×3) color palette. The former representation is called *an RGB image*, whereas the latter is called *an indexed image*.
- *Pseudocolor image processing techniques* assign color to pixels based on an interpretation of the data rather than the original scene color (which may not even be known). *Full-color image* processing methods, on the other hand, process the pixel values of images whose colors usually correspond to the color of the original scene.

WHAT HAVE WE LEARNED?

- *Several monochrome image processing techniques, from edge detection to histogram equalization, can be extended to color images.*
- The success of applying such techniques to color images depends on the choice of color model used to represent the images.

PROBLEMS

Problem 1. Use the MATLAB function `patch` to display the *RGB* cube in Figure 11.9.

Problem 2. Write MATLAB code to add two RGB color images. Test it with two test images (of the same size) of your choice. Are the results what you had expected?

Problem 3. What is wrong with the following MATLAB code to add an indexed color image to a constant for brightening purposes? Fix the code to achieve the desired goal.

```
[X, map] = imread('canoe.tif');  
X = X + 20;
```


PROBLEMS

Problem 4. In our discussion of pseudocoloring, we stated that the intensity slicing method described in Section 11.4.1 is a particular case of the more general method (using transformation functions) described in Section 11.4.2. Assuming that a 256-level monochrome image has been “sliced” into four colors, red, green, blue, and yellow, and that each range of gray levels has the same width (64 gray levels), plot the (staircase-shaped) transformation functions for each color channel (R, G, and B).



PROBLEMS

Problem 5. Use the edge function in MATLAB and write a script to compute and display the edges of a color image for the following cases:

- (a) RGB image, combining the edges from each color channel by adding them up.
- (b) RGB image, combining the edges from each color channel with a logical OR operation.
- (c) YIQ image, combining the edges from each color channel by adding them up.
- (d) YIQ image, combining the edges from each color channel with a logical OR operation.



END OF LECTURE XI

LECTURE 12 – IMAGE COMPRESSION AND CODING

